

Axon: A Typed, Stage-Validated Model Compiler for Neural Network Programs

Brainsurgery contributors

May 11, 2026

Abstract

Axon is a domain-specific language and compiler pipeline for describing neural network architectures in a form that remains inspectable, typed, and close to the parameter structure of modern checkpoints. The current implementation is organized as a sequence of explicit source-to-source and source-to-graph stages. Each stage consumes a single canonical Axon abstract syntax tree or a typed graph representation, strengthens a local invariant, and exposes those invariants to validation and round-trip tests. This document describes the system in the style of a conference systems paper: we state the design goals, define each pipeline contract, and identify the boundary between frontend normalization, typed core construction, graph lowering, and backend execution.

1 Introduction

Large language model implementations are usually distributed as a mixture of Python modules, checkpoint-specific configuration conventions, and tensor state dictionaries. This ecosystem is flexible, but it makes independent analysis, transformation, and backend generation difficult. A compiler that targets model families rather than one hand-written implementation per checkpoint must preserve several kinds of information simultaneously: lexical names, symbolic dimensions, parameter paths, optional configuration defaults, control-flow structure, and precise tensor types.

Axon addresses this problem with a small model-description language and a compiler pipeline that deliberately avoids string-serialization round trips in semantic stages. The central design choice is that the pre-lowering compiler uses one canonical Axon AST. Stages do not switch to unrelated syntax trees; instead, they progressively remove surface syntax, resolve names, elaborate implicit structure, flatten control, and attach inferred metadata. Graph lowering then translates the fully typed flat program into an in-memory Graph IR consumed by code generation and runtime backends.

The goal of this document is not to specify all user-facing Axon syntax. It is to make the current pipeline legible as a compiler system: what each stage accepts, what it guarantees, what it refuses to do, and why those boundaries matter for correctness and backend portability.

Contributions. This document makes three system contributions. First, it gives a stage contract for the active Axon compiler path, from source text to typed Graph IR. Second, it describes how typed metadata and symbolic shape information flow through the compiler without relying on rendered strings as semantic data. Third, it defines the backend boundary used by torch, pipeline, runtime, and tinygrad execution targets.

2 Design Goals

Single semantic AST. All pre-lowering stages operate on the same Axon AST representation. A parsed, resolved, normalized, flattened, and typed program differs by invariants and metadata, not by changing the meaning of nodes through ad hoc intermediate formats.

Stage-local contracts. Each stage owns a narrow contract. For example, name resolution does not infer tensor types; flattening does not repair type errors; backends do not interpret surface Axon scope syntax. This separation allows validators and round-trip tests to target individual compiler properties.

No model-family special casing in the compiler core. Model-specific quirks belong in loading and integration paths, not in parsing, type checking, lowering, graph IR, code generation, runtime, or builtins. Compiler stages must implement generic language semantics.

Typed graph execution. Backends consume a structured, typed Graph IR rather than YAML-like dictionaries or string-rendered call forms. This preserves shape and type information for future backend optimizations and reduces accidental semantic drift.

3 The Axon Language

Axon is designed as a compact functional language for model-family descriptions. Its first-class domain objects are tensors, symbolic dimensions, paths into a checkpoint state dictionary, configuration values, and reusable library definitions. The language is intentionally biased toward describing the mathematical structure of a neural network rather than the mechanics of a specific Python implementation.

Tensors and dimensions first. Tensor types carry symbolic shapes. A type such as

`Tensor[B,S,D]`

states that a value is a tensor whose batch, sequence, and hidden dimensions are represented by the dimension symbols B, S, and D. Dimension symbols are not comments: they participate in typechecking, unification, primitive type rules, and Graph IR metadata. This lets a reusable definition express shape relationships without hard-coding checkpoint sizes.

Haskell-inspired syntax. Definitions use signatures, curried-looking arrows, optional parameters, and a `do`-style statement form:

```
block :: Tensor[B,S,D] -> ?Tensor[B,K] -> Tensor[B,S,D]
block x attn_mask = do
  h <- NN.rmsnorm@ln x
  a <- Attention.attention h h h attn_mask
  return x + a
```

The syntax is not Haskell, but it borrows the idea that signatures are central documentation and that sequencing should expose dataflow explicitly. Pipes and scoped parameter paths are surface conveniences that are normalized away before flattening.

Functional semantics. Axon definitions are pure functions over explicit arguments, configuration lookups, and parameter-path reads. A `bind` statement names the value produced by an expression. Except for ternary branch expressions, expressions are eager: a call used as an argument is evaluated before the callee receives the value. This eagerness is important for flattening and for preserving the semantics of computations with expensive tensor operations.

Paths and reusable libraries. Checkpoint parameters are accessed through typed path values. Relative path sugar is lexical authoring syntax; the flat core uses ordinary `Path` arguments and absolute or templated absolute paths. Reusable library modules such as `NN`, `Tensor`, `Attention`, `Masking`, `MoE`, `Cache`, and `Positions` define model-independent operations. Model files combine these libraries with checkpoint-specific configuration paths and architecture-level control.

Semantic core. The compiler treats the surface language as notation for a smaller core. The core contains explicit calls, explicit optional/default arguments, explicit path arguments, flat bindings, helper definitions for loops, and typed primitive calls. Backend implementations consume Graph IR derived from this core; they do not implement surface Axon syntax.

4 Type System, Checking, and Inference

The active type system is implemented by `typecheck2`. It is not a separate backend contract and it is not the removed legacy checker. Its input is a flat, closed Axon AST after normalization, elaboration, and flattening; its output is the same AST with typed module signatures and per-expression metadata. The checker is MAIN-module based: it resolves the effective MAIN module, computes the reachable definition graph, prunes unreachable definitions, checks demanded definitions from MAIN, and validates the typed result.

Types. The implemented type language includes `Any`, `Bool`, `Dim`, `Float`, `Int`, `Null`, `Path`, `String`, `List [T]`, tuples, optionals `?T`, tensor types

$$\text{Tensor}[d_1, \dots, d_n],$$

tensor sub-bases such as `IdxTensor[...]`, named type aliases, and generated type variables. Dimension tokens may be integer literals, symbolic names, variadic binders such as `...S`, or symbolic expressions over `+`, `-`, `*`, and `/`. Type aliases are expanded during unification and normalization, including aliases with variadic dimension parameters.

Expression metadata. The typing judgment can be read operationally as

$$\Gamma \vdash e \Rightarrow (e^\tau, \tau, a, d),$$

where Γ contains term bindings, desugared `Path` bindings, dimension-name bindings, module definitions, aliases, expression aliases, type-variable substitutions, and dimension substitutions. The result is a typed expression e^τ , inferred type τ , inferred arity a , and inferred tensor dimensions d when applicable. Typed validation rejects missing arity metadata and rejects tensor expressions without inferred dimensions.

Literals, ascriptions, and numeric inference. Variables are typed by environment lookup. Unbound names are type errors. Literals initially receive their natural scalar types: integers are `Int`, floats are `Float`, booleans are `Bool`, strings are `String`, null is `Null`, and path literals are `Path`. Type ascriptions ($e :: T$) are checked rather than erased: the inferred type of e is unified with T , and the ascription remains in the typed AST. Numeric unification is deliberately pragmatic: `Int` with `Float` yields `Float`, `Dim` with `Float` yields `Float`, and `Int` with `Dim` yields `Dim`. Numeric literals are retagged in typed contexts, e.g. integer literals inside `List [Dim]` shape arguments become dimension literals.

Unification and dimension names. Unification first applies substitutions and expands aliases. Type variables bind with an occurs check; `Any` unifies with and yields the other side; lists, tuples, optionals, named types, and tensor types unify structurally. Tensor unification checks compatible bases and dimension sequences. The generic base `Tensor` can unify with more specific tensor bases, while incompatible non-generic bases are rejected. Dimension unification supports ordinary symbols, integer literals, variadic dimension binders, and symbolic binary expressions. It simplifies common arithmetic identities, rejects recursive dimension bindings, and prefers readable existing names over generated names when two dimensions are equivalent.

Calls and call-site specialization. User definitions, builtin Axon definitions, and generated helpers all use the same call path. At a call site, the callee signature is instantiated with fresh type and dimension variables, explicit path arguments are consumed first, actual arguments are typed, and actual types are unified with formal parameter types. Dimension-typed actual expressions are also used to bind fresh signature dimensions to source-level dimension names. When safe, `typecheck2` checks the callee body under the call-site environment and uses the specialized body return type if it is more precise than the declared header. Caller substitutions are restored afterward so local inference does not leak arbitrarily across calls. Recursive calls use the current instantiated interface when the callee is already active.

Primitive rules. Primitive calls are registered in `brainsurgery/synapse/ops`. Each primitive provides `OP_NAME` and a `LOWERING_TYPE_SIGNATURE`; it may additionally provide a `type_rule` hook. `Typecheck2` checks actuals against the lowering signature, rejects explicit `null` where a non-optional primitive parameter is required, then lets the primitive rule compute a more precise result type when necessary. This is where rules for shape-transforming operations such as `reshape`, `expand`, `permute`, `transpose`, `chunk`, `split`, `concat`, and `slice` live, as well as neural-network primitives such as `embedding`, `linear`, `expert_linear`, `layernorm`, and `rmsnorm`.

Branches, nullability, and returns. Ternaries and expression-level `if` require boolean conditions. Branch types are joined structurally: tuples itemwise, optionals by joining inner types, and tensors by preserving equivalent dimensions or using broadcasting where appropriate. Null checks refine environments: in a branch guarded by `x == null` or `x != null`, a value of type `?T` is treated as `Null` on the null side and `T` on the non-null side. Return and yield statements are checked against declared return types when available, while preserving protected header dimension names so literals or generated call-site names do not overwrite meaningful signature dimensions.

Destructuring and generated helpers. Multi-target binds destructure tuples of matching arity, lists by repeating the list item type, and unconstrained type variables by binding them to tuples of fresh type variables. Generated loop and branch helpers are ordinary definitions: they do not have a compatibility typing mode. After checking, generated helper signatures and body annotations are normalized and canonicalized. `Typecheck2` repeats the pass up to a fixed bound and returns once the typed AST reaches a deterministic fixpoint; short equivalent-name cycles are broken by choosing a deterministic structural spelling.

Current limits. The active checker does not expose a complete public constraint store over dimensions, integers, booleans, and nullability. Some broad primitive signatures still rely on primitive-specific hooks for precision. Unknown keyword arguments on user calls are still permissively carried by the checker, although later validation or lowering may reject them. These are implementation facts, not intended backend escape hatches.

5 System Model

An Axon compilation unit consists of model-family source files, builtin Axon definitions, checkpoint metadata, and a selected main definition. The compiler must support both generic model-family files and materialized checkpoint variants. A model definition may reference symbolic dimensions, configuration values, optional cache values, parameter paths, scoped parameter templates, and builtin neural-network operations.

The compiler has two semantic representation families. The first is the Axon AST used by all frontend and typed-core stages. The second is Graph IR, a typed SSA-like representation inspired by standard compiler dataflow representations such as SSA [Cytron et al., 1991]. Flattening uses an administrative-normal-form style decomposition of nested expressions into explicit binds, similar in spirit to ANF/CPS compiler normal forms [Sabry and Felleisen, 1992]. Backend execution currently targets PyTorch [Paszke et al., 2019] and `tinygrad`.

6 Compiler Invariants

The most important invariant is monotonicity of structure: later stages should see fewer surface constructs and more metadata, never weaker semantic information. For example, after resolution, there should be no unresolved imports; after normalization, there should be no callee path sugar; after flattening, there should be no statement-level control; after `typecheck2`, graph lowering should not need to infer missing tensor types.

The second invariant is backend isolation. Backend implementations may choose different tensor libraries and primitive implementations, but they must consume the same Graph IR contract. If a backend requires weaker validation or model-family-specific dispatch to run, the issue belongs either in the backend implementation or in an explicitly approved language extension.

source Axon $\rightarrow$ parse/load/materialize $\rightarrow$ resolve/validate $\rightarrow$ normalize/elaborate $\rightarrow$ flatten $\rightarrow$
 typecheck2 $\rightarrow$ Graph IR $\rightarrow$ backend execution

Figure 1: Default Axon compilation path. Optional optimization is excluded from the correctness-critical path in this paper.

7 Pipeline Overview

The current default path is shown below. Optional optimization is separated from the correctness-critical path; the backend path is expected to work without the AST optimizer.

Phase	Representation	Primary invariant
Parse/load/materialize	one-file ASTs	syntactic structure and checkpoint defaults
Resolve/validate	closed AST	no unresolved imports or names
Normalize/elaborate	normalized AST	explicit call and default structure
Flatten	flat AST	no statement-level control or relative paths
Typecheck2	typed flat AST	expression types, arities, and dimensions
Graph lowering	Graph IR	typed SSA-like graph modules
Backend	generated/runtime code	executable tensor program

The following sections describe the stages in order.

8 Parse

Input and output. The parse stage consumes Axon source text and produces a one-file Axon AST. It records syntactic structure, source-level names, declarations, imports, exports, pragmas, type ascriptions, path literals, and expression forms.

Contract. Parsing is intentionally syntax-local. It does not load imported files, resolve names, infer types, normalize call syntax, materialize configuration values, or lower operations. A parse result may contain unresolved references and surface sugar; those are obligations of later stages.

Rationale. Keeping parse pure makes it possible to test grammar stability independently from the file system and from checkpoint availability. It also prevents the parser from becoming an implicit compatibility layer for downstream stages.

Formal view. Let Σ be source text and A_f be a one-file Axon AST. Parse is a partial function

$$\text{parse} : \Sigma \rightharpoonup A_f.$$

It is undefined for syntactically invalid programs, but it is defined for programs with unresolved identifiers, unresolved imports, or type errors.

Example. The source fragment

```
import NN

embed :: Tensor[B,S] -> Tensor[B,S,D]
embed ids = NN.embedding@wte ids dim=D
```

is parsed into an import declaration, a typed definition, a parameter list, and a call expression whose callee still carries surface path sugar `@wte`. Parse records this structure but does not decide whether `NN.embedding` exists or whether `D` is a valid dimension symbol.

9 Load

Input and output. The load stage consumes a root one-file AST plus origin-path information and returns the root program together with the imported Axon files needed by that program.

Contract. Loading is responsible for file discovery and import-root policy. It does not merge modules, rewrite references, prune unreachable definitions, or infer types. The output is still a collection of one-file ASTs.

Rationale. Separating loading from parsing keeps source-text parsing deterministic and lets resolution operate over an explicit import set. This also makes it easier to test imported builtins and model files without hiding file-system effects in the parser.

Formal view. For a root AST A_0 , an origin path p , and an import search policy $\mathcal{I}$, loading computes a finite import closure:

$$\text{load}(A_0, p, \mathcal{I}) = [A_0, A_1, \dots, A_n].$$

The result is ordered only for deterministic processing; it is not yet a single linked program.

Example. If a model file imports

```
import NN
import Attention (attention, reshape_heads)
```

load finds the corresponding builtin Axon files and parses them. The model AST still contains import declarations, and the imported ASTs remain separate files. Qualified names such as `Attention.attention` are not resolved until the next stage.

10 Materialize

Input and output. Materialization consumes a generic one-file Axon AST and checkpoint metadata. It emits a one-file Axon AST specialized enough to be inspected or used as a checkpoint-specific artifact.

Contract. The stage may resolve materialization-specific configuration expressions, but it must not become an optimizer. It does not perform dead-code elimination, general constant folding, alias expansion, inlining, type inference, or backend-specific rewriting.

Rationale. Generic model files are the source of truth for model families. Materialized files are useful for inspection and targeted debugging, but materialization must not silently repair or reinterpret the program in ways that diverge from the generic source.

Formal view. For generic AST A_f and checkpoint metadata C , materialization is a source-to-source transformation

$$\text{materialize}(A_f, C) = A'_f$$

that may replace materialization-approved configuration expressions with checkpoint-specific values. It must preserve the program’s definition graph and must not depend on backend selection.

Example. A generic constant may be written as

```
MODEL_DIM = Config.int@@hidden_size default=768
```

For a checkpoint whose configuration contains `hidden_size = 1024`, materialization may emit

```
MODEL_DIM = 1024
```

It should not also inline all uses of `MODEL_DIM`, delete unused branches, or specialize unrelated functions.

11 Resolve

Input and output. Resolution consumes the loaded set of one-file ASTs and produces a single closed Axon AST.

Contract. Resolve merges imported modules, resolves qualified and unqualified references, checks export visibility, removes import-surface state, identifies the main module, and prunes definitions that are unreachable from the main module under the current compilation mode.

Exclusions. Resolve does not typecheck expressions, infer dimensions, lower operations, or apply warning policy. Diagnostics such as unused imports or strict warning promotion belong to validation or orchestration.

Rationale. Downstream stages should never need to search the file system or infer lexical scope from import declarations. They receive one closed program with explicit definition identities.

Formal view. Resolution maps a finite set of one-file ASTs to one closed AST:

$$\text{resolve} : [A_f] \rightarrow A_c.$$

The function is undefined when imports are missing, exported names are ambiguous, or reachable expressions reference undefined names. The output A_c has no import-surface state in the reachable program.

Example. Given

```
import Tensor (reshape)
```

```
f x = reshape x shape=[B, S, D]
```

resolve links `reshape` to the imported `Tensor.reshape` definition. If both `Tensor.reshape` and another imported `reshape` are available unqualified, resolution reports ambiguity rather than choosing one.

12 Validate

Input and output. Validation consumes a closed AST and returns the same AST when the requested invariant set holds. On failure, it reports diagnostics rather than producing a repaired program.

Contract. The pre-flatten validator checks closedness, unresolved-name rejection, type-alias arity and visibility, and structural properties required before normalization and flattening. Later validators check stronger stage-specific properties such as normalized calls, flat control, backend-required graph shape, and typed metadata consistency.

Rationale. Validation is a certification layer, not a transformation layer. This is what lets later stages reject invalid inputs instead of weakening their assumptions.

Formal view. A validator is a predicate over an AST representation:

$$\text{validate}_k(A) = \begin{cases} A & \text{if invariant set } k \text{ holds,} \\ \perp & \text{otherwise.} \end{cases}$$

The returned AST is structurally identical to the input. Diagnostics are separate from transformation.

Example. Before flattening, validation rejects a closed program that still contains a reachable unresolved name:

```
f x = does_not_exist x
```

After flattening, a stronger validator rejects statement-level control:

```
if cond then do
  y <- f x
else do
  y <- g x
```

because flat Axon must express this through values, helpers, or ternaries.

13 Normalize

Input and output. Normalize consumes a validated closed AST and emits a normalized closed AST.

Contract. The stage removes call and control sugar that would otherwise complicate flattening. Callee path suffixes become ordinary **Path**-typed arguments, pipe syntax becomes ordinary calls, callable-name expressions that denote zero-argument calls become explicit calls, and loop bodies receive explicit yield/carry protocol structure.

Exclusions. Normalize does not globally expand default arguments. Defaults are handled by the elaboration stage and by type-aware binding where appropriate.

Rationale. Normalize is the boundary between surface authoring syntax and a regularized AST. It keeps later stages from duplicating parser-level sugar handling.

Formal view. Normalize is a total transformation over validated closed ASTs:

$$\text{normalize} : A_c \rightarrow A_n.$$

It preserves denotation while reducing the number of syntactic forms. The validator for A_n checks that eliminated sugar does not reappear.

Example: path and pipe sugar. The surface expression

```
x <- input |> NN.layernorm@ln_f eps=1e-5
```

normalizes to an ordinary call whose path is an explicit argument:

```
x <- NN.layernorm @ln_f input eps=1e-5
```

The exact rendered form may differ, but the AST no longer represents either a pipe expression or callee-attached path sugar.

Example: loop protocol. A source loop with implied carry/yield is normalized so that the loop body has an explicit final yield. Flattening therefore sees one loop protocol rather than several authoring conveniences.

14 Elaborate

Input and output. Elaboration consumes a normalized closed AST and produces a closed AST with implicit call structure made explicit where the language contract requires it.

Contract. The current elaboration stage focuses on default and optional argument structure needed by downstream stages. It preserves user-visible type ascriptions and does not erase inferred metadata. It is mandatory in the default pipeline because later stages should not need to guess whether an omitted optional argument was intentional or simply absent from the surface syntax.

Rationale. Elaboration sits after normalization because normalization first determines which expressions are calls. Once calls are explicit, elaboration can apply signature-aware rules without reinterpreting syntax.

Formal view. For normalized AST A_n , elaboration produces A_e :

$$\text{elaborate} : A_n \rightarrow A_e.$$

The transformation is signature-aware but not type-inferential: it uses declared defaults and optional parameters to make call structure explicit, but it does not solve tensor shapes.

Example. If a library function has the signature

```
linear :: Path -> Tensor[B,S,D] -> ?Dim -> ?Bool -> Tensor[B,S,0]
linear path x ?dim=null ?bias=true = ...
```

then a call such as

```
y <- NN.linear @w x
```

can be elaborated to an equivalent call with explicit defaulted arguments:

```
y <- NN.linear @w x dim=null bias=true
```

This prevents flattening and lowering from needing to reason about omitted arguments.

15 Flatten

Input and output. Flatten consumes an elaborated closed AST and emits a flat closed Axon AST.

Contract. The flat program has no statement-level `if`, no `for`, no `scope`, no relative paths, and no hidden callee path sugar. Complex expressions are lowered into explicit bind statements, loop constructs are converted into helper definitions, and returns/yields are restricted to atomic variables or tuples of variables.

Path semantics. Flatten resolves lexical scopes into explicit absolute or templated absolute paths. Scoped callees receive ordinary `Path`-typed parameters rather than syntactic path-sugar parameters.

Rationale. Flatten produces the small core language expected by typecheck2 and Graph IR lowering. Backends should not implement surface Axon control constructs.

Formal view. Flatten maps elaborated closed ASTs to flat closed ASTs:

$$\text{flatten} : A_e \rightarrow A_{\text{flat}}.$$

It preserves evaluation order by introducing fresh bind names. If $\text{atoms}(e)$ are variables, literals, paths, or tuples of atoms, then flat bind right-hand sides are calls or primitive expressions over atoms.

Example: expression flattening. The nested source expression

```
y <- NN.linear @w (Tensor.reshape x shape=[B, S, D]) dim=0
```

becomes a sequence such as

```
_v1 <- Tensor.reshape x shape=[B, S, D]
y <- NN.linear @@w _v1 dim=0
```

where the generated temporary preserves eager evaluation order.

Example: scope and loop lowering. A scoped loop

```
x <- for@h i <- [0..NUM_LAYERS) carry (x) do
  x <- scope@attn do
    NN.linear@c_proj x
  yield x
```

is converted into helper definitions and calls with ordinary path parameters. The resulting paths are absolute templates such as `@@'h.{i}.attn.c_proj'` rather than dynamic scope statements.

16 Typecheck2

Input and output. Typecheck2 consumes a flat closed AST and emits a fully typed flat closed AST.

Contract. Every expression receives inferred type and arity metadata. Tensor types carry symbolic shape variables, optional types preserve nullability, and primitive calls are checked against declared signatures plus primitive-specific type rules. Function calls are checked through signature binding and unification, including call-site specialization where the definition contract permits it.

Constraints. The type system records symbolic relationships over dimensions, integers, booleans, and nullability when they are available from the program structure. These constraints are metadata for later validation and optimization; they do not license backends to guess missing types.

Rationale. Typed flat Axon is the last AST-level representation before Graph IR. Its metadata must be strong enough that lowering does not need compatibility fallbacks or model-specific inference.

Formal view. Typecheck2 is a judgment over flat definitions:

$$\Gamma; \Phi \vdash e : \tau, a, \Delta$$

where Γ maps term names to types, Φ stores known symbolic constraints, τ is the inferred type, a is arity, and Δ contains inferred dimension metadata. A checked program is a flat AST whose expression nodes are annotated with these results.

Example: tensor unification. If

```
tok :: Tensor[B,S,D]
pos :: Tensor[B,S,D2]
x <- tok + pos
```

then the primitive rule for addition requires compatible tensor shapes. The typechecker unifies `D2` with `D` and annotates the result as `Tensor[B,S,D]`. This avoids leaving downstream stages with unrelated fresh dimensions for tensors that must be equal.

Example: optional cache types. A value of type

```
?CacheLayer[B,H,T,DH]
```

is either `null` or a cache layer with the stated tensor shape. Typecheck2 preserves that inner shape through null checks and cache-update primitives so that graph lowering can retain cache metadata.

17 Graph IR Lowering

Input and output. Graph lowering consumes a flat, closed, fully typed Axon AST and produces an in-memory typed Graph IR program.

Contract. The lowering stage translates Axon definitions to graph modules, bind statements to graph nodes, multi-output expressions to structured graph outputs, and path literals to structured graph paths. It preserves operand types, output types, constraints, constants, and module boundaries needed by backends and later graph-level analysis.

Exclusions. Lowering does not accept surface constructs such as `scope`, `for`, or statement-level `if`. It also does not rewrite primitive calls into legacy string-based callee forms.

Rationale. Graph IR replaces the old serialized graph contract. The compiler should pass typed structured objects to backends rather than relying on YAML-shaped dictionaries or render/parse cycles.

Formal view. Lowering is a homomorphism from typed flat modules to graph modules:

$$\text{lower} : A_{\text{flat}}^{\tau} \rightarrow G.$$

Each flat definition becomes a graph module. Each flat bind becomes one graph node with typed operands and typed outputs. Calls to user definitions become module-call nodes; calls to primitive wrappers lower to primitive graph ops only when the source definition is itself a primitive wrapper.

Example. A typed flat bind

```
tok <- NN.embedding @@wte input_ids dim=MODEL_DIM
```

lowers to a graph node whose op is the embedding primitive, whose first operand is a structured absolute path `@@wte`, and whose output type is the inferred tensor type, e.g. `Tensor[B,S,MODEL_DIM]`.

Multi-output nodes. Graph IR preserves multi-output structure. A flat destructuring bind such as

```
q, k, v <- Tensor.chunk qkv parts=3
```

can lower to a single graph node with three outputs rather than a synthetic list plus three unrelated index operations.

18 Graph Validation

Input and output. Graph validation consumes a Graph IR program and either certifies it or reports a structural error.

Contract. The validator checks module inputs and outputs, operand availability, definition reachability, type compatibility between actual operands and formal parameters, graph path structure, and backend-relevant invariants such as the absence of unsupported surface constructs.

Rationale. Graph validation is the last generic compiler guard before backend-specific execution. It should catch lowering bugs before they become silent numerical divergences in generated code.

Formal view. Graph validation checks a graph program $G = (M, E)$ against a collection of well-formedness predicates:

$$\text{validateGraph}(G) = \text{defs}(G) \wedge \text{types}(G) \wedge \text{uses}(G) \wedge \text{paths}(G).$$

The validator is intentionally stricter than backend code generation: a graph that fails validation should not be emitted for any backend.

Example. If lowering emits a node whose formal parameter expects `Tensor[B,S,D]` but whose actual operand is annotated as `Float`, graph validation fails before torch or tinygrad code is generated. Similarly, if a node references a value not produced by module inputs or earlier nodes, validation reports an undefined operand.

19 Graph Execution Backends

Input and output. Execution backends consume Graph IR and a checkpoint state dictionary and produce model outputs. The active backend family includes `codegen2-torch`, `runtime2-torch`, `pipeline2-torch`, and `codegen2-tinygrad`.

Torch code generation. `codegen2-torch` emits direct Python/PyTorch code. It is the primary fidelity baseline for compiler-stage changes because it has the broadest operator coverage and the closest numerical behavior to the reference Transformers implementation.

Runtime interpretation. `runtime2-torch` interprets Graph IR without emitting a Python source module. It is useful for debugging graph semantics and for separating graph correctness from code-emission bugs.

Pipeline execution. `pipeline2-torch` partitions Graph IR for multi-device execution. Its contract is graph-based; it should not depend on old Synapse YAML artifacts or surface Axon scope constructs.

Tinygrad code generation. `codegen2-tinygrad` is a separate backend that targets tinygrad. Backend fixes should be tinygrad-local unless they expose a generic Graph IR or Axon stage bug. It must not weaken earlier validation contracts to make unsupported patterns run.

Formal view. An execution backend implements an interpretation function

$$\mathcal{E}_B(G, \theta, c, x) = y$$

for backend B , state dictionary θ , configuration c , and model inputs x . Fidelity testing compares $\mathcal{E}_B$ against the reference implementation for the same checkpoint.

Example. For a graph node representing

```
_linear @@w x weight_path=@weight bias_path=@bias
```

the torch backend emits a call to a generated helper that reads `w.weight` and optional `w.bias` from the state dictionary and applies PyTorch linear algebra. The tinygrad backend emits analogous tinygrad tensor operations but must preserve the same path lookup and transpose semantics.

20 Validation and Round Trips

The implementation treats rendering as an observability mechanism, not as a semantic transport. Stage tests therefore distinguish weak and strong round-trips. A weak round-trip reparses the rendered output and reruns only the minimum stage subset needed to compare the stage artifact. A strong round-trip reruns all earlier stages before comparing the artifact. Strong round-trips are more expensive, but they catch hidden dependencies on unrendered state and stale inferred metadata.

The same philosophy applies to Graph IR rendering. The graph may be rendered for inspection or debugging, but execution backends receive the in-memory graph object. This avoids the class of failures caused by serializing typed operands to strings and parsing them back into weaker representations.

21 Failure Isolation

The staged architecture is intended to make failures actionable. A closedness failure is a resolver or validation issue; a rank mismatch before lowering is a typecheck2 issue; a Graph IR type compatibility error is a lowering or graph validator issue; a torch-only numerical mismatch after Graph IR validation is usually an Axon semantic or primitive type-rule issue; a tinygrad-only mismatch is normally a backend implementation issue. This classification is an engineering discipline rather than a theorem, but it is essential for avoiding model-specific fixes in compiler layers.

Set	Families	Axon files	Pairs	Test pairs
All model files	54	256	433	31
Generic model-family files	54	77	300	31

Table 1: Declared Axon model coverage in the repository snapshot.

Run	Completed	Errors	Top-1 failures	Strict diffs	Policy issues
tinygrad max-1B snapshot	64	6	0	3	0
tinygrad max-1B earlier sweep	92	40	0	16	14
tinygrad targeted parity rerun	11	0	0	2	2

Table 2: Representative fidelity snapshots. The current snapshot is from run `tinygrad-max1b-current-serial-20260511-120638`. “Strict diffs” counts $\max |\Delta| \geq 10^{-3}$. “Policy issues” excludes accepted random test-checkpoint rows with top-1 agreement and $\max |\Delta| < 10^{-1}$.

22 Backend Scope

The active backends are:

- `codegen2-torch`, a direct Python/PyTorch code generator from Graph IR;
- `runtime2-torch`, an interpreter for the same graph contract;
- `pipeline2-torch`, a graph partitioning backend for multi-device execution;
- `codegen2-tinygrad`, a newer tinygrad code-generation backend.

The tinygrad backend is intentionally separate and configurable; it should improve coverage and speed through backend-local implementations rather than by weakening earlier compiler invariants.

23 Experimental Evaluation

The practical evaluation target is checkpoint fidelity: for each Axon file and checkpoint pair, compare the Axon-derived model against the reference Transformers model using masked top-1 agreement and masked maximum absolute logit difference. Torch parity is required for any change that affects Axon source files or compiler stages up to Graph IR. Backend-only changes may be evaluated against an already validated Graph IR artifact and then swept across model families.

The test matrix should report at least three categories: healthy rows, hard execution errors, and numerical divergences above the chosen threshold. Test checkpoints with random weights are useful for feature coverage, but their fidelity thresholds are interpreted separately from real checkpoints.

Coverage. Table 1 summarizes the model coverage declared in the repository snapshot used for this document. A “pair” is one Axon file and one declared checkpoint. Test checkpoints are synthetic random models used to exercise families whose real checkpoints are too large for routine smoke testing.

Fidelity snapshots. Table 2 reports representative benchmark snapshots from the active Graph IR path. The tinygrad row is a current serial max-1B run snapshot; it was still running when the table was generated. Under the project policy used here, test-checkpoint rows with top-1 agreement and maximum absolute difference below 10^{-1} are not treated as blocking divergences.

max-1B snapshot	completed rows by outcome	
healthy/policy-ok		58
error		6

Figure 2: Outcome distribution for the current tinygrad max-1B snapshot (64 completed rows).

24 Related Work

Axon follows common compiler practice by separating parsing, name resolution, normalization, type checking, and graph lowering. Its flat core is close in spirit to ANF-style intermediate representations [Sabry and Felleisen, 1992], while Graph IR adopts the single-assignment and dataflow flavor of SSA-based compiler representations [Cytron et al., 1991]. Its execution backends are designed to interoperate with tensor libraries such as PyTorch [Paszke et al., 2019]. The distinguishing requirement is not merely graph execution, but preserving model-family parameter paths, symbolic dimensions, and checkpoint configuration semantics through the compiler.

25 Discussion

The staged design trades implementation simplicity in a single monolithic compiler pass for stronger local reasoning. This matters for model compilers because many failures are otherwise ambiguous: a wrong logit can arise from a bad model file, a type inference bug, a path-resolution bug, a lowering bug, a backend numerical mismatch, or a checkpoint-loading quirk. Stage contracts make these failure modes easier to isolate.

The main open design pressure is where to place optimization. Some transformations are clearly source-level simplifications, while others are better expressed on Graph IR with explicit dataflow and typed operands. The current contract keeps optimization optional and requires the backend path to remain correct without it. This makes optimization a performance and readability layer rather than a hidden correctness dependency.

26 Conclusion

Axon is evolving toward a compiler architecture in which every transformation has a named contract, every backend consumes typed structured data, and model family quirks are isolated from compiler semantics. The current pipeline is already capable of resolving model-family Axon programs, flattening and typing them, lowering them to Graph IR, and executing them through multiple backends. The remaining work is to harden validation coverage, improve typed artifact round-tripping, and make backend-specific performance improvements without reintroducing implicit compatibility paths.

References

- Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, 1991.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.
- Amr Sabry and Matthias Felleisen. Reasoning about programs in continuation-passing style. In *Proceedings of the ACM Conference on Lisp and Functional Programming*, 1992.